Pre-Race Physiology → Race Performance

Goal

Investigate whether an athlete's physiology and training load in the weeks **before** a race predict their race outcome. We want to answer questions like:

- Does higher average HRV in the 2 weeks before a race correlate with a better finish position?
- Does sleeping more (or less) before a race matter?
- Is there a "sweet spot" for training load (TSS) before a race?
- Can we build a simple model that predicts finish position from pre-race metrics?

Background

We track daily metrics for each athlete from TrainingPeaks:

Metric	What it is
HRV	Heart Rate Variability — higher generally means better recovery
RHR	Resting Heart Rate — lower generally means better fitness
Sleep hours	Self-reported sleep duration
TSS	Training Stress Score per workout — how hard a single session was

> **Note on CTL / ATL / TSB:** These are powerful training load metrics (fitness, fatigue, form) but they are currently **empty** for most athletes in our database because TrainingPeaks restricts them to premium accounts. The export includes the columns, but expect them to be null. Focus on the metrics that *do* have data: **HRV, RHR, sleep, TSS, workout count, and total training hours per day**. Training hours and workout count are excellent proxies for training load — every athlete has them.

We also have race results from World Triathlon with finish position, split times, and event metadata.

The Two Tables

You'll work with two CSV files exported from our database.

Table 1: `races.csv` — One row per race

Column	Description
`athlete_id`	Unique athlete identifier
`athlete_name`	Athlete's name
`event_date`	Date of the race (YYYY-MM-DD)
`event_name`	Name of the race (e.g., "2025 World Cup Viña del Mar")
`race_category`	Event tier: `wtcs`, `worldcup`, `continental_cup`, `other`
`distance`	Race distance: `sprint`, `standard`, `super_sprint`, etc.
`finish_status`	`FINISH`, `DNF`, `DNS`, `DSQ`
`finish_position`	Integer finish place (null if DNF/DNS/DSQ)
`field_size`	Number of athletes who started the race
`finish_pct`	`finish_position / field_size` — normalized so you can compare across field sizes

Table 2: `daily_metrics.csv` — One row per athlete per day

Column	Description
`athlete_id`	Unique athlete identifier
`date`	Calendar date (YYYY-MM-DD)

Column	Description
`sleep_hours`	Hours of sleep
`rhr`	Resting heart rate (bpm)
`hrv`	Heart rate variability
`ctl`	Chronic Training Load (fitness)
`atl`	Acute Training Load (fatigue)
`tsb`	Training Stress Balance (form = CTL − ATL)
`num_workouts`	Number of workouts that day
`total_tss`	Sum of TSS across all workouts that day
`total_duration_hrs`	Total training hours that day

Your Task — Step by Step

Phase 1: Data Exploration

1. **Load the CSVs** into a Jupyter notebook using `pandas`.
2. **Filter to Blake Bullard**: `races = races[races["athlete_id"] == 13]` and same for metrics. Start with one athlete.
3. **Inspect the data**: `.head()`, `.describe()`, `.info()`, check for nulls.
4. **Visualize distributions**: histograms of HRV, sleep, RHR, finish_position, daily training hours.
5. **Filter**: Remove rows where `finish_status != 'FINISH'` (DNFs aren't useful for position analysis).
6. **Plot his race timeline**: finish_position over time (x=event_date, y=finish_position). Does he have good and bad stretches?

Phase 2: Build the Analysis Dataset

This is the key step — you need to **join** the two tables by computing pre-race averages.

For each race, compute the average of each daily metric over a **lookback window** (e.g., 14 days before the race). Copy this into a notebook cell and run it:

```
import pandas as pd

races = pd.read_csv("races.csv", parse_dates=["event_date"])
metrics = pd.read_csv("daily_metrics.csv", parse_dates=["date"])

# Filter to Blake Bullard
races = races[races["athlete_id"] == 13].copy()
metrics = metrics[metrics["athlete_id"] == 13].copy()

# Only finished races
races = races[races["finish_status"] == "FINISH"].copy()

lookback_days = 14  # Try 7, 14, 21, 28 later

rows = []
for _, race in races.iterrows():
    window_start = race["event_date"] - pd.Timedelta(days=lookback_days)
    window_end = race["event_date"] - pd.Timedelta(days=1)  # Don't include race day

    # Filter metrics for this athlete in this window
    mask = (
        (metrics["athlete_id"] == race["athlete_id"])
        & (metrics["date"] >= window_start)
        & (metrics["date"] <= window_end)
    )
    window = metrics[mask]

    if len(window) < 3:  # Skip if too little data
        continue

    rows.append({
        "athlete_id": race["athlete_id"],
        "event_date": race["event_date"],
        "event_name": race["event_name"],
        "race_category": race["race_category"],
        "finish_position": race["finish_position"],
        "finish_pct": race["finish_pct"],
        "field_size": race["field_size"],
```

```
        # Pre-race averages
        "avg_hrv": window["hrv"].mean(),
        "avg_rhr": window["rhr"].mean(),
        "avg_sleep": window["sleep_hours"].mean(),
        "avg_daily_tss": window["total_tss"].mean(),
        "avg_daily_hours": window["total_duration_hrs"].mean(),
        "avg_num_workouts": window["num_workouts"].mean(),
        "days_with_data": len(window),
    })

analysis_df = pd.DataFrame(rows)
```

Phase 3: Visual Analysis

Create scatter plots and see if patterns emerge. Copy and run:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Scatter: HRV vs finish percentage
sns.scatterplot(data=analysis_df, x="avg_hrv", y="finish_pct")
plt.xlabel("Avg HRV (14d before race)")
plt.ylabel("Finish Position %  (lower = better)")
plt.title("Does higher HRV predict better race results?")
plt.gca().invert_yaxis()  # Lower % = better
plt.show()
```

Repeat for: `avg_sleep`, `avg_rhr`, `avg_daily_tss`, `avg_daily_hours`, `avg_num_workouts`.

Correlation matrix:

```
cols = ["finish_pct", "avg_hrv", "avg_rhr", "avg_sleep", "avg_daily_tss", "avg_daily_hours", "avg_num_workouts"]
sns.heatmap(analysis_df[cols].corr(), annot=True, cmap="coolwarm", center=0)
plt.title("Correlation Matrix")
plt.show()
```

Phase 4: Regression

Try a simple linear regression. Copy and run:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import LeaveOneOut
from sklearn.metrics import mean_absolute_error
import numpy as np

features = ["avg_hrv", "avg_rhr", "avg_sleep", "avg_daily_tss", "avg_daily_hours", "avg_num_workouts"]
X = analysis_df[features].dropna()
y = analysis_df.loc[X.index, "finish_pct"]

# Leave-one-out cross validation (good for small datasets)
loo = LeaveOneOut()
predictions = []
actuals = []

for train_idx, test_idx in loo.split(X):
    model = LinearRegression()
    model.fit(X.iloc[train_idx], y.iloc[train_idx])
    pred = model.predict(X.iloc[test_idx])
    predictions.append(pred[0])
    actuals.append(y.iloc[test_idx].values[0])

print(f"MAE: {mean_absolute_error(actuals, predictions):.3f}")
print(f"Correlation: {np.corrcoef(actuals, predictions)[0,1]:.3f}")
```

Important Considerations

Use `finish_pct` not `finish_position`

Finishing 5th out of 10 is very different from 5th out of 60. Dividing position by field size normalizes this. Always use `finish_pct` as your target variable.

Try multiple lookback windows

Don't just use 14 days. Try 7, 14, 21, 28 and see which window has the strongest correlation. This is a real finding — does recent sleep matter more than a month of sleep?

Watch out for confounders

- **Athlete ability**: A great athlete might have good HRV *and* good results, but one doesn't cause the other — both come from being fit. Consider analyzing **within-athlete** variation (how an athlete's HRV differs from their own average) rather than only across athletes.
- **Race tier**: World-level races have tougher fields. Include `race_category` as a control variable.
- **Sample size**: With few athletes and races, correlation ≠ causation. Be honest about this in your write-up.

Within-athlete analysis (stretch goal)

Instead of raw values, compute **deviations from the athlete's own baseline**:

```
# Per-athlete z-score
for col in ["avg_hrv", "avg_sleep", "avg_rhr"]:
    grouped = analysis_df.groupby("athlete_id")[col]
    analysis_df[f"{col}_zscore"] = grouped.transform(lambda x: (x - x.mean()) / x.std())
```

This asks: "When *this* athlete sleeps more than *their* usual, do *they* perform better?" — which is a much stronger claim than comparing across athletes.

Training load tapering is your most interesting question

Coaching wisdom says athletes should **taper** before a race — reduce training volume/intensity in the final days while maintaining fitness built up over weeks. Look for this by comparing the athlete's average daily TSS in the **7 days** before a race vs. the **28 days** before. If the ratio is < 1 (they trained less the final week), that's a taper. Does tapering correlate with better results?

```
# Compute a simple taper ratio per race
analysis_df["taper_ratio"] = analysis_df["avg_tss_7d"] / analysis_df["avg_tss_28d"]
# < 1.0 means they eased off; > 1.0 means they pushed harder
```

This requires building the analysis dataset with *two* lookback windows per race (7d and 28d). It's extra work but it's a much more interesting finding than a simple average.

Getting Started

1. Generate the CSV files

Run this from the `PodiumDashboard/` directory:

```
python scripts/export_physiology_data.py
```

This creates two files in `outputs/` : - `outputs/races.csv` - `outputs/daily_metrics.csv`

2. Know your data

Not all athletes have the same coverage. Here's a summary from the latest export:

Athlete	Races	Workout-Days	HRV-Days	Best for...
Blake Harris	24	498	486	**Full analysis** (HRV + training + races)
Blake Bullard	16	507	491	**Full analysis**
Keller Norland	22	444	0	Training load → race only (no physiology)
Reese Vannerson	19	508	5	Training load → race only
Sullivan Middaugh	20	48	0	Races only (limited training data)

Start with Blake Bullard (athlete_id=13) — he has the richest combined data (16 races, 507 workout-days, 491 HRV-days). Get the full analysis working for one athlete first, then extend to others. Blake Harris (id=95) is your next best candidate. After that, athletes without HRV can still be analyzed using training load proxies (daily hours, workout count, TSS).

3. Start your notebook

Create a new Jupyter notebook (e.g., `notebooks/physiology_experiment.ipynb`) and begin with Phase 1.

4. Libraries you'll need

```
pip install pandas matplotlib seaborn scikit-learn jupyter
```

Deliverables

1. A Jupyter notebook with your analysis (all phases)
2. At least 3 visualizations (scatter plots, correlation matrix, etc.)
3. Answers to: Which lookback window worked best? Which metrics correlated most with finish position? Did the regression model add value over simple correlations?

Questions to Think About

- Is 14 days the right lookback? What if the night before matters more than 2 weeks ago?
- Should you weight recent days more heavily than older days? (Exponential moving average vs. simple average)
- Does the relationship change by race category? (WTCS vs. Continental Cup)
- What happens if you add **trends** (is HRV going up or down?) instead of just averages?
- Could you detect if an athlete is "peaking" vs. "fatigued" before a race?
- After finishing the analysis for Blake Bullard, do the same patterns hold for Blake Harris? For athletes with only training load data (no HRV)?